

RELATÓRIO E PROPOSTA TÉCNICA

Plataforma de Documentação Corporativa

Modelo reutilizável para estruturar documentação de arquitetura, operação e usabilidade em sistemas digitais.

Dimensão	Definição
Público técnico	Arquitetura, integrações, segurança, dados, implantação e manutenção.
Público usuário	Tutoriais orientados a tarefas, permissões, resultados esperados e solução de problemas.
Portal	Docusaurus para organizar, publicar, pesquisar e versionar o conteúdo.
Automação	Playwright para navegar no sistema, validar jornadas e produzir evidências visuais.
Publicação	Estratégia definida caso a caso: embutida no produto, deploy separado ou domínio próprio.

Origem deste documento: O modelo descrito aqui não é teórico: foi aplicado integralmente em uma implantação real, e as recomendações práticas, armadilhas e critérios de validação apresentados derivam dessa experiência.

1. Resumo executivo

A proposta consiste em transformar o conhecimento de um sistema em um produto de documentação próprio, com navegação semelhante aos grandes portais técnicos. Em vez de manter informações dispersas em arquivos, mensagens e conhecimento informal da equipe, a organização passa a ter uma fonte de consulta estruturada para desenvolvedores, suporte, administradores e clientes.

O modelo separa o conteúdo em duas vertentes principais. A primeira explica como o sistema é construído e operado. A segunda ensina como utilizá-lo. Essa separação reduz ruído: o cliente não precisa compreender detalhes de banco de dados e o desenvolvedor encontra decisões técnicas sem atravessar tutoriais básicos.

Resultado esperado: Uma documentação com identidade visual do produto, busca, menu lateral, diagramas, tutoriais ilustrados, histórico de versões e processo claro de atualização — publicada em um endereço estável e com rotina de manutenção definida.

2. Problema que a iniciativa resolve

- Conhecimento crítico concentrado em poucas pessoas.
- Onboarding lento de desenvolvedores, suporte e clientes.
- Tutoriais desatualizados depois de mudanças de interface.
- Ausência de visão consolidada sobre arquitetura, permissões, integrações e operação.
- Respostas diferentes para a mesma dúvida, dependendo de quem atende.
- Dificuldade de demonstrar maturidade técnica, segurança e governança do produto.

3. Objetivos

Objetivo geral

Implantar uma plataforma de documentação corporativa sustentável, integrada ao ciclo de desenvolvimento e aplicável a qualquer sistema web.

Objetivos específicos

- Organizar a documentação técnica e funcional em uma estrutura previsível.
- Criar tutoriais orientados a tarefas, com linguagem adequada ao usuário final.
- Padronizar diagramas, screenshots, glossário, alertas e referências.
- Automatizar a captura de telas e a validação dos fluxos documentados.
- Permitir revisão, versionamento e publicação junto ao código-fonte.
- Definir uma estratégia de publicação compatível com a infraestrutura existente.
- Reduzir o custo de suporte, treinamento e transferência de conhecimento.

4. Visão da solução

A solução utiliza documentação como código. Os artigos são escritos em Markdown ou MDX, revisados no mesmo processo do software e publicados como um portal web. O conteúdo permanece independente do

sistema principal, mas pode usar a identidade visual e evidências capturadas em um ambiente de demonstração.

Camada	Responsabilidade	Exemplos
Conteúdo	Registrar conhecimento técnico e funcional.	Artigos, guias, referências, FAQ e glossário.
Apresentação	Transformar os arquivos em um portal navegável.	Menu lateral, busca, tema claro/escuro e versionamento.
Evidência visual	Gerar imagens consistentes e reproduzíveis.	Login, dashboard, formulários, relatórios e configurações.
Validação	Confirmar que jornadas e páginas continuam operacionais.	Testes de navegação, permissões, URLs e elementos esperados.
Publicação	Levar o portal a um endereço acessível e estável.	Build estática, hospedagem e automação de deploy.
Governança	Definir responsáveis, revisão e critérios de publicação.	Pull requests, revisão periódica e matriz de cobertura.

4.1 Adaptação à identidade visual do produto

O portal deve transmitir continuidade visual com o produto documentado, sem reproduzir sua interface integralmente. A documentação continua sendo um ambiente de leitura e consulta: a identidade da marca orienta cores, tipografia, logotipo, bordas e estados de interação, enquanto fundos neutros, espaçamento confortável e hierarquia editorial preservam a legibilidade.

Princípio: O portal deve parecer parte do mesmo ecossistema, mas permanecer independente. A documentação recebe uma cópia controlada dos tokens e assets aprovados; não importa estilos diretamente do código do produto nem depende dele para executar.

Elemento	O que identificar no produto	Aplicação no portal
Tokens e tema	Cores primária, secundária, neutras, semânticas e modo escuro.	Links, foco, item ativo, botões, alertas e superfícies.
Tipografia	Famílias, pesos, escala, altura de linha e contraste.	Corpo dos artigos, títulos, navegação e blocos técnicos.
Componentes	Raios, bordas, sombras e estados de interação.	Cards, busca, tabelas, paginação e menus.
Marca	Logotipo, símbolo, versões clara/escura e área de proteção.	Navbar, página inicial, favicon e metadados.
Layout	Densidade, largura, espaçamento e comportamento responsivo.	Conteúdo, sidebar, navbar e navegação móvel.

Processo recomendado

1. Auditar o design existente. Ler arquivos de tema, estilos globais, configuração de design e componentes estruturais, registrando a origem de cada decisão visual.

2. Criar um inventário de tokens. Mapear valores brutos para papéis semânticos: primária, superfície, borda, texto, texto secundário, sucesso, alerta e erro.
3. Definir temas claro e escuro. Manter o mesmo significado para cada token nos dois temas, ajustando luminosidade e contraste em vez de apenas inverter cores.
4. Transportar assets aprovados. Copiar logotipo e ícones institucionais para a estrutura da documentação, evitando referências às pastas internas do produto.
5. Personalizar o gerador. Aplicar os tokens às variáveis de tema, priorizando navbar, sidebar, busca, links, cards, tabelas, código, alertas e rodapé.
6. Validar e registrar. Revisar contraste, responsividade, foco de teclado e leitura nos dois temas. Documentar tokens, arquivos de origem e exceções.

Cuidado com a tipografia da marca

Nem todo token do produto deve ser transportado literalmente. Fontes de exibição — condensadas, decorativas ou aplicadas em caixa alta — costumam funcionar bem em painéis e títulos curtos, mas prejudicam a leitura de textos longos. Nesses casos, recomenda-se preservar a identidade pela cor, pelo logotipo e pelos componentes, mantendo uma fonte de leitura confortável no corpo dos artigos. A decisão deve ser registrada para não ser interpretada como esquecimento.

Critérios de aceitação visual

- Paleta e tipografia derivadas de fontes confirmadas no produto, sem cores inventadas.
- Logotipo copiado para a documentação e adequado aos fundos claro e escuro.
- Cor primária concentrada em ações, links, foco e pequenos destaques; grandes áreas permanecem confortáveis para leitura.
- Contraste suficiente entre texto, fundo, links, bordas e estados de foco.
- Navbar, sidebar, busca, cards, tabelas, alertas, código, breadcrumbs e paginação coerentes.
- Tema claro e escuro revisados em desktop, tablet e celular.
- Navegação por teclado visível e componentes sem perda de informação por uso exclusivo de cor.
- Build independente e ausência de referências de execução às pastas do produto.

5. O que é o Docusaurus

Docusaurus é um gerador de sites voltado à publicação de documentação. Ele converte arquivos Markdown/MDX em um portal web e oferece uma base pronta para navegação, menus laterais, temas, internacionalização, pesquisa, páginas institucionais e versionamento de documentos.

Na prática, ele é a camada de apresentação da solução. Os autores escrevem arquivos de texto simples; o Docusaurus organiza esses arquivos, aplica o layout e gera uma versão estática que pode ser publicada em serviços de hospedagem convencionais.

Principais contribuições

- Estrutura consistente de categorias, artigos e conteúdos relacionados.
- Suporte a Markdown e MDX, permitindo texto, componentes e exemplos técnicos.
- Personalização de cores, tipografia, logotipo, navegação e modo escuro.
- Versionamento para manter documentação alinhada a diferentes releases.
- Integração com mecanismos de busca e suporte a múltiplos idiomas.

- Renderização de diagramas Mermaid quando o recurso é habilitado.
- Geração de arquivos estáticos, simplificando hospedagem, desempenho e segurança.

Importante: O Docusaurus não analisa o sistema nem descobre sozinho como ele funciona. Ele organiza e publica o conteúdo produzido pela equipe ou por processos assistidos.

6. O que é o Playwright

Playwright é uma ferramenta de automação de navegadores e testes de ponta a ponta. Ela controla Chromium, Firefox e WebKit por código, reproduzindo ações reais como abrir páginas, autenticar, clicar, preencher campos, aguardar respostas e verificar resultados.

Nesta proposta, o Playwright cumpre duas funções. Primeiro, produz screenshots padronizados para os tutoriais. Segundo, valida os caminhos descritos na documentação, ajudando a identificar páginas removidas, mudanças de rótulo, falhas de navegação ou resultados diferentes do esperado.

Aplicações recomendadas

- Reutilizar um estado autenticado de uma conta exclusiva de demonstração.
- Abrir rotas e módulos em viewport padronizado.
- Aguardar estados reais da interface em vez de depender de pausas fixas.
- Mascaram nomes, e-mails, telefones e identificadores sensíveis.
- Capturar screenshots, vídeos e traces para diagnóstico.
- Comparar imagens de referência para detectar regressões visuais.
- Executar testes isolados e reproduzíveis em integração contínua.

Importante: O Playwright não é a plataforma de documentação. Ele automatiza o sistema documentado e fornece validação e evidências que serão utilizadas pelo portal.

7. Como as ferramentas se complementam

Ferramenta	Papel principal	O que entrega	O que não substitui
Docusaurus	Publicação e navegação	Portal, artigos, menus, busca e versões	Análise do código e automação do sistema
Playwright	Automação e validação	Testes, capturas, vídeos, traces e evidências	Portal, redação e governança editorial
Mermaid	Representação visual	Diagramas mantidos como texto junto aos artigos	Modelagem formal ou descoberta automática
Git / CI	Governança e entrega	Histórico, revisão e publicação automatizada	Definição de conteúdo e responsáveis

Fluxo resumido: a equipe analisa o sistema, redige os artigos, adiciona diagramas, executa o Playwright para validar jornadas e atualizar imagens, revisa as alterações e publica o portal gerado pelo Docusaurus.

8. Estrutura editorial recomendada

8.1 Arquitetura — equipe técnica

- Visão geral, objetivos e limites do sistema.
- Stack, organização do frontend e serviços de backend.
- Modelo de dados, entidades, relacionamentos e migrações.
- Autenticação, autorização, perfis, permissões e isolamento de dados.
- APIs, integrações externas, webhooks e processamento assíncrono.
- Fluxos críticos, regras de negócio e decisões arquiteturais.
- Segurança, auditoria, observabilidade e tratamento de falhas.
- Configuração, variáveis de ambiente, build, deploy e rollback.
- Limitações conhecidas, riscos e troubleshooting.

8.2 Usabilidade — cliente e operação

- Primeiros passos, acesso, perfis e navegação.
- Tutoriais organizados por objetivo do usuário.
- Pré-requisitos e permissões necessárias.
- Passos numerados com imagens nos pontos decisivos.
- Resultado esperado e como confirmar que a ação funcionou.
- Alertas sobre impactos, restrições e ações irreversíveis.
- Problemas comuns e caminhos de recuperação.
- Perguntas frequentes e próximos passos.

8.3 Referência

- Glossário de termos funcionais e técnicos.
- Matriz de perfis e permissões.
- Catálogo de módulos, rotas, APIs e integrações.
- Políticas de segurança e privacidade aplicáveis.
- Histórico de versões e alterações relevantes.
- Matriz de cobertura entre funcionalidade, artigo e screenshot.

9. Estrutura de diretórios reutilizável

A documentação vive fora dos produtos, em uma instalação compartilhada que gera o portal de vários sistemas a partir de uma única base de dependências. Cada produto contribui apenas com o próprio conteúdo e a própria identidade; a ferramenta é comum a todos.

documentacao/	(instalação compartilhada, fora dos produtos)
├─ node_modules/	(uma vez só – as dependências são pesadas)
├─ compartilhado/	(tema base, CSS, componentes e utilitários de captura)
├─ projetos/	
│ └─ <produto>/	
│ └─ docs/	(arquitetura, usabilidade, referência)
│ └─ static/img/	(marca e capturas geradas)
│ └─ playwright/	(login e specs de captura do produto)
│ └─ tema.css	(identidade visual do produto)
│ └─ projeto.json	(nome, endereço, seções e repositório)

```
|— scripts/                (build, publicação, índice e painel)
|— painel/                (painel local de operação)
|— docusaurus.config.ts
|— playwright.config.ts
|— sidebars.ts
|— package.json
```

A instalação é única porque as dependências do gerador e da automação são pesadas — facilmente centenas de megabytes. Replicá-las em cada produto desperdiça espaço e multiplica as atualizações. Aqui existe um só conjunto de dependências, e a configuração escolhe qual produto montar por uma variável de ambiente.

Um painel local concentra a operação: escolher o produto, criar um novo a partir do repositório dele, gerar as capturas, gerar o build, publicar e excluir — acompanhando o log. Cada produto é descrito em um arquivo de projeto, que informa nome, endereço, seções, identidade visual e onde fica o repositório de destino nesta máquina.

Publicação: o comando de publicação de um produto gera o build e o copia para o diretório público do repositório daquele produto. Apenas o build atravessa a fronteira; a fonte, as dependências e as credenciais permanecem na instalação compartilhada e nunca vão para o produto.

Observação: Pipelines de integração contínua devem residir no diretório de workflows da raiz do repositório. Arquivos de workflow colocados dentro da pasta da documentação não são reconhecidos pela plataforma e nunca serão executados.

10. Método de implantação

1. Descoberta. Mapear módulos, rotas, perfis, integrações, dados e fluxos críticos. Registrar evidências e lacunas.
2. Fundação. Criar ou reutilizar a instalação compartilhada, adicionar o produto, instalar o gerador e a automação, configurar idioma, navegação, temas e scripts.
3. Inventário técnico. Produzir mapas de componentes, rotas, dados, permissões e integrações antes de escrever explicações definitivas.
4. Arquitetura. Redigir conteúdo técnico com origem verificável, diagramas e limites conhecidos.
5. Usabilidade. Documentar tarefas reais a partir das telas, permissões e validações existentes.
6. Automação visual. Criar uma conta de demonstração e roteiros de autenticação, navegação e capturas.
7. Revisão. Auditar links, imagens, cobertura, segurança, consistência e aderência ao sistema.
8. Publicação. Definir a estratégia de hospedagem, gerar a build, publicar e validar o resultado no endereço final.

Ordem importa: O inventário técnico precede a redação porque reduz o risco mais comum deste tipo de projeto: produzir uma documentação bem apresentada, porém desconectada do sistema real.

11. Estratégias de publicação e hospedagem

A documentação gerada é um site estático: um conjunto de arquivos HTML, CSS, JavaScript e imagens. Para que qualquer pessoa a acesse, esses arquivos precisam ser entregues por um servidor web. Não existe cenário em que o portal fique disponível sem estar hospedado em algum lugar; o que varia é onde os arquivos residem e sob qual endereço são publicados.

Esta seção descreve as três estratégias possíveis, seus custos reais e os critérios para escolher entre elas. A decisão depende principalmente de dois fatores: o grau de controle sobre a hospedagem do produto e a tolerância a acoplamento entre documentação e aplicação.

11.1 Opção A — Publicação embutida no produto

O resultado do build da documentação é copiado para o diretório de arquivos estáticos da aplicação (por exemplo, a pasta `public` em projetos Vite) e segue junto no deploy do produto. O portal passa a responder em um subcaminho do mesmo domínio.

- **Endereço resultante:** `https://app.exemplo.com/docs`
- **Quando faz sentido:** a hospedagem do produto é fechada e não permite regras de proxy ou rewrite; a equipe não quer criar nova infraestrutura, domínio ou registro de DNS.
- **Requisitos:** configurar o caminho base do gerador e versionar o artefato de build no repositório do produto, já que a plataforma publica a partir do repositório.

Custos e riscos

- Acopla artefatos gerados ao repositório do produto, que cresce a cada republicação.
- Depende de o servidor priorizar arquivos reais sobre o redirecionamento padrão de aplicações de página única.
- Atualizar a documentação exige republicar o produto inteiro.

Regra crítica: Copiar apenas o diretório de build. Mover o projeto de documentação inteiro para dentro da pasta pública publicaria dependências, código-fonte e arquivos de ambiente — incluindo credenciais — em endereço acessível pela internet. Em um caso real, essa pasta somava centenas de megabytes e continha um arquivo de ambiente com senha de teste.

11.2 Opção B — Deploy separado em subdomínio

O build é publicado em um serviço de hospedagem de sites estáticos e um registro de DNS aponta um subdomínio institucional para esse serviço. A documentação passa a ter ciclo de vida próprio, independente da aplicação.

- **Endereço resultante:** `https://docs.exemplo.com`
- **Quando faz sentido:** há preferência por isolamento total, a documentação tem cadência de atualização própria ou deseja-se evitar o crescimento do repositório do produto.
- **Requisitos:** acesso à zona de DNS do domínio e uma conta em serviço de hospedagem estática. O caminho base do gerador permanece na raiz.

Esclarecimento frequente: Um subdomínio pertence ao mesmo domínio institucional, mas constitui um host distinto. Ele não precisa estar no mesmo servidor da aplicação — e é justamente essa separação

que elimina conflitos de rota. Roteamento por DNS distingue nomes de host, nunca caminhos; publicar em um subcaminho depende necessariamente do servidor da aplicação.

11.3 Opção C — Rota na aplicação carregando o portal

Cria-se uma rota dentro da aplicação que incorpora o portal por meio de um quadro embutido. É a alternativa que aparenta maior integração, mas raramente compensa.

Por que evitar

- A documentação continua precisando estar hospedada em outro endereço; o quadro apenas a exibe.
- Prejudica links diretos, rolagem, impressão, indexação e acessibilidade.
- Exige alteração no código do produto, criando manutenção adicional.

Deve ser considerada apenas quando houver exigência explícita de manter o usuário dentro da aplicação durante a consulta.

11.4 Quadro comparativo

Critério	A — Embutida	B — Separada	C — Quadro embutido
Endereço	app.exemplo.com/docs	docs.exemplo.com	app.exemplo.com/ajuda
Infraestrutura nova	Nenhuma	Hospedagem e DNS	Hospedagem e alteração no app
Acoplamento	Alto (artefato no repositório)	Nenhum	Médio
Atualizar a documentação	Republicar o produto	Publicar apenas a documentação	Publicar apenas a documentação
Risco técnico	Conflito com rotas da aplicação	Baixo	Experiência de uso e links diretos
Peso no repositório	Cresce a cada publicação	Não afeta	Não afeta

11.5 Configuração conforme o cenário

A escolha do endereço determina dois parâmetros do gerador. Configurá-los incorretamente produz links quebrados e imagens ausentes, mesmo com o conteúdo correto.

- **Publicação em subcaminho (Opção A):** definir o caminho base como o subcaminho desejado. Quando o site inteiro é a documentação, ajustar também o caminho das rotas de documentos para a raiz, evitando endereços duplicados como /docs/docs/.
- **Publicação na raiz ou subdomínio (Opção B):** manter o caminho base na raiz.
- **Links internos:** são relativos ao caminho base. Ao migrar para um subcaminho, referências absolutas escritas manualmente precisam ser revisadas.

```
// Publicação em subcaminho
url: 'https://app.exemplo.com',
baseUrl: '/docs/',
presets: [['classic', { docs: { routeBasePath: '/' } }]]

// Publicação em subdomínio
```

```
url: 'https://docs.exemplo.com',  
baseUrl: '/',
```

Reversibilidade: Migrar de uma estratégia para outra exige apenas ajustar esses parâmetros e republicar. Recomenda-se registrar a decisão e o caminho de reversão em um documento de publicação, para que a escolha inicial não se torne irreversível na prática.

11.6 Como validar a publicação

Esta é a etapa mais frequentemente executada de forma incorreta. Aplicações de página única respondem com sucesso a praticamente qualquer endereço, devolvendo o arquivo principal da aplicação. Verificar apenas o código de status HTTP produz falso positivo: a resposta indica 200, mas o conteúdo entregue é a aplicação, não a documentação.

Procedimento recomendado

1. Requisitar um arquivo com extensão, como o mapa do site ou uma imagem. Arquivos com extensão não são capturados pelo redirecionamento da aplicação: se retornarem erro, os arquivos não foram publicados.
2. Conferir o conteúdo de uma página interna, verificando o título. Se aparecer o título da aplicação, o endereço está sendo tratado pela aplicação e não pela documentação.
3. Acessar diretamente uma página interna, e não apenas a inicial, para validar links profundos.
4. Confirmar o carregamento de imagens, folhas de estilo e diagramas.
5. Repetir a verificação após cada mudança de estratégia de publicação.

Lição de campo: Em uma implantação real, as rotas da documentação retornaram status 200 mesmo sem terem sido publicadas; apenas a inspeção do título revelou que o conteúdo servido era a aplicação. O indício decisivo foi uma imagem retornando erro, o que comprovou que os arquivos não estavam no servidor.

11.7 Barra final nas URLs

Geradores de site estático produzem cada página como um arquivo de índice dentro de uma pasta com o nome da página. Sem a barra final, muitos servidores não resolvem esse índice: a URL deixa de corresponder a um arquivo, cai no redirecionamento da aplicação e o usuário recebe a página de erro do produto — mesmo com a documentação corretamente publicada.

URL solicitada	Resultado
/docs/	Página inicial da documentação
/docs/secao/artigo/	Artigo correto
/docs/secao/artigo	Página de erro da aplicação

Recomendações: habilitar a opção de barra final no gerador, para que sitemap e links internos fiquem consistentes; e garantir que toda URL construída fora do gerador — links na aplicação, índices de busca, materiais de divulgação — termine com barra.

Atenção: Habilitar a barra final altera a resolução de links relativos que apontam para diretórios. É esperado que a verificação de links do build acuse ocorrências existentes; corrija-as apontando para o arquivo de destino em vez do diretório.

11.8 Limitação do servidor de desenvolvimento

Servidores de desenvolvimento de aplicações de página única costumam redirecionar toda requisição HTML para a aplicação, sem resolver índices de diretório. Na prática, a documentação publicada em subcaminho ****não abre**** durante o desenvolvimento, embora funcione no build de produção.

Isso confunde equipes e gera diagnósticos equivocados. A verificação local deve ser feita com o build de produção ou com o servidor do próprio gerador de documentação. Vale registrar essa limitação no material de operação para evitar retrabalho.

12. Automação de publicação e manutenção

A publicação manual funciona nas primeiras semanas e falha no médio prazo, porque depende de memória. Recomenda-se automatizar o que é mecânico e sinalizar o que exige julgamento humano.

O que automatizar

Automação	Comportamento	Benefício
Publicação	Ao alterar a origem da documentação, executar o build e publicar o resultado.	Elimina o passo manual e a divergência entre o escrito e o publicado.
Lembrete de revisão	Ao alterar código do sistema, sinalizar as áreas de documentação afetadas.	Ataca a causa mais comum de defasagem: o esquecimento.
Artefatos derivados	Regenerar mapa de rotas, catálogo de serviços e variáveis de ambiente a partir do código.	Mantém automaticamente correta a parte mecânica do conteúdo.
Regressão visual	Reexecutar capturas e comparar com imagens de referência.	Detecta mudanças de interface que invalidam tutoriais.

Onde publicar o lembrete: adapte ao fluxo da equipe

O lembrete de revisão precisa aparecer onde a equipe efetivamente trabalha. Times que revisam alterações antes da integração recebem melhor um comentário na solicitação de alteração; times que integram diretamente no ramo principal precisam do aviso no próprio commit. Escolher a variante errada produz uma automação que nunca dispara — falha silenciosa e difícil de perceber.

Fluxo da equipe	Onde avisar	Observações
Usa solicitações de alteração (pull requests)	Comentário na solicitação, antes da integração	Melhor momento: permite corrigir a documentação junto com a mudança. Comentar apenas uma vez por solicitação, evitando repetição a cada envio.
Integra direto no ramo principal	Comentário no próprio commit	Notifica quem fez a alteração. Exige mais cuidado com ruído: não avisar quando a própria documentação já foi alterada no mesmo envio, nem nos commits gerados pela publicação automática.
Cadência baixa ou equipe pequena	Verificação periódica agregada	Uma execução semanal compara alterações do sistema e da documentação e abre um único

Fluxo da equipe	Onde avisar	Observações
		registro de pendência. Menor ruído, resposta mais lenta.

Independentemente da variante, o lembrete ganha muito valor ao indicar ****quais**** páginas provavelmente foram afetadas, mapeando diretórios do código para artigos da documentação. Um aviso genérico é ignorado; um aviso que aponta o arquivo a revisar costuma ser atendido.

Permissões: Automações que comentam ou gravam no repositório precisam de permissão de escrita, geralmente desabilitada por padrão. Vale confirmar esse ajuste antes de concluir que a automação está com defeito.

Cuidados de implementação

- Não acoplar o build da documentação ao build do produto: as dependências do gerador são pesadas e uma falha na documentação passaria a interromper o deploy da aplicação.
- Quando o pipeline gravar o artefato no repositório, restringir os gatilhos aos caminhos de origem, evitando que a própria publicação reative o processo indefinidamente.
- Pipelines que gravam no repositório exigem permissão de escrita, normalmente desabilitada por padrão.
- Capturas automatizadas exigem credenciais de demonstração armazenadas como segredos, nunca no repositório.

Limite da automação: Texto explicativo e tutoriais não se atualizam sozinhos. Nenhuma automação identifica que uma regra de negócio mudou e reescreve o parágrafo correspondente. A automação reduz esforço mecânico e evita esquecimento; a correção do conteúdo permanece responsabilidade humana.

13. A documentação como fonte da ajuda no produto

Sistemas costumam ter uma tela de ajuda com conteúdo escrito diretamente no código: perguntas frequentes fixas, textos que envelhecem e ninguém revisa. Isso duplica conhecimento e cria duas versões da verdade — a da documentação e a da tela.

Com a documentação estruturada, essa tela pode deixar de ter conteúdo próprio e passar a consumir a documentação. O ganho é duplo: acaba a duplicação e a busca da ajuda passa a alcançar todo o acervo, não apenas as poucas perguntas cadastradas.

13.1 Como funciona

1. Durante a publicação, um script percorre os artigos e gera um índice em formato JSON com título, seção, endereço, títulos internos e texto limpo de cada página, além das perguntas frequentes já estruturadas.
2. Esse índice é publicado como arquivo estático, junto da documentação.
3. A aplicação busca o arquivo em tempo de execução e renderiza a lista de perguntas e os resultados de busca.

Sem acoplamento: A aplicação não importa nada do projeto de documentação: apenas consome um arquivo estático. Os dois continuam com dependências e builds independentes, preservando a regra definida no início deste documento.

13.2 Busca por conteúdo, não apenas por pergunta

A busca deixa de filtrar uma lista fixa e passa a pesquisar o acervo inteiro. Ao digitar um termo como "card", o usuário recebe tanto as perguntas relacionadas quanto os artigos que tratam do assunto, com um trecho do texto e link direto para a página.

Cuidados de implementação

- Ignorar acentos e maiúsculas na comparação: em português, o usuário digita sem acento e espera encontrar o termo acentuado. Sem isso, buscas legítimas retornam vazio.
- Pontuar por relevância: ocorrência no título vale mais que em título de seção, que vale mais que no corpo do texto.
- Normalizar o texto uma única vez ao carregar o índice, e não a cada tecla digitada.
- Exibir o trecho ao redor da ocorrência, cortando em limites de palavra para não truncar no meio.
- Construir os endereços com barra final, pelos motivos descritos na seção de publicação.

13.3 Filtragem por perfil

Nem todo artigo interessa a todo usuário: documentação de arquitetura e telas administrativas não deveriam aparecer para um usuário comum. A solução é marcar o público-alvo no próprio artigo, em um campo do cabeçalho, e filtrar índice e busca conforme o perfil de quem consulta.

Essa marcação pode ser derivada de informação que os tutoriais já possuem — a seção que descreve quem pode executar a tarefa —, o que a torna consistente com o conteúdo em vez de ser mais um dado a manter à parte.

Limite importante: Esse filtro organiza a experiência de leitura; não é controle de acesso. O índice é um arquivo estático e público. Conteúdo verdadeiramente sensível não deve estar na documentação publicada.

13.4 Comportamento em caso de falha

Se o índice não puder ser carregado, a tela deve exibir um aviso curto com link para a documentação completa. A tentativa de manter uma cópia do conteúdo no código como reserva recria exatamente a duplicação que a integração eliminou.

14. Padrões de autoria

Tipo de artigo	Estrutura mínima
Arquitetura	Objetivo; contexto; componentes; fluxo; regras; segurança; falhas; observabilidade; limitações; conteúdos relacionados; data da última revisão.
Tutorial	O que será feito; quem pode fazer; pré-requisitos; passos; resultado esperado; observações; problemas comuns; próximos passos.
Referência	Definição; sintaxe ou campos; restrições; exemplos; erros; relação com outros recursos.
Troubleshooting	Sintoma; possíveis causas; diagnóstico; correção; validação; quando escalar.

Toda afirmação importante deve ser confirmada no comportamento atual do produto. Quando algo for inferido, planejado ou ainda não validado, essa condição deve aparecer explicitamente no texto.

Documentar apenas o que está ativo: Recursos desativados, ocultos ou inacessíveis no produto não devem ser descritos como disponíveis. Ao encontrar um módulo desativado, a decisão correta é registrar a condição — ou omitir o tutorial — em vez de documentar um comportamento que o usuário não conseguirá reproduzir.

15. Estratégia para screenshots

- Utilizar ambiente de demonstração com dados fictícios e estáveis.
- Definir viewport, tema, idioma e navegador para manter consistência.
- Desabilitar animações e aguardar elementos por estado semântico.
- Usar seletores estáveis, preferindo identificadores a textos traduzíveis.
- Mascaram dados pessoais e identificadores antes de salvar a imagem.
- Capturar somente telas que ajudam o usuário a decidir ou confirmar uma ação.
- Nomear arquivos por módulo e tarefa, evitando nomes genéricos.
- Reexecutar capturas quando mudanças de interface afetarem o tutorial.

Detalhes técnicos que costumam ser descobertos tarde

Situação	Recomendação
Rotas presumidas	Extrair as rotas do roteador do produto. Nomes intuitivos frequentemente não correspondem às rotas reais, e o roteiro falha silenciosamente ao capturar páginas de erro.

Situação	Recomendação
Idioma do navegador	Navegadores automatizados não assumem o idioma do usuário final. Fixar o idioma na configuração; caso contrário, a interface é capturada em outro idioma e seletores baseados em texto deixam de funcionar.
Tela de login	Capturas autenticadas reaproveitam a sessão salva. A tela de login exige um contexto sem sessão, do contrário a captura resulta na área interna.
Tabelas largas	A captura de página inteira estende a altura, não a largura. Conteúdos mais largos que a janela aparecem cortados; ajustar a largura ao conteúdo antes de capturar.
Mascaramento amplo	Aplicar desfoque a todas as imagens atinge logotipo e ilustrações. Mascaram seletivamente e privilegiar dados fictícios como proteção principal.
Imagens ainda inexistentes	Referenciar imagens não geradas pode interromper o build. Configurar o gerador para emitir aviso enquanto as capturas não existirem.

Segurança: O estado autenticado contém informações de sessão. Esse arquivo não deve ser versionado nem compartilhado, e a conta utilizada deve possuir apenas as permissões necessárias. Antes do primeiro commit, confirmar que arquivos de ambiente, sessões e dependências estão fora do controle de versão.

16. Vídeos-tutorial de jornada

Além das capturas estáticas, o Playwright grava a própria página em vídeo enquanto percorre a aplicação. Isso permite produzir tutoriais curtos que mostram uma tarefa completa do início ao fim — por exemplo, criar um projeto, adicionar pessoas ao time e criar cards em um quadro Kanban. O recurso já está disponível e reutiliza o mesmo ambiente de demonstração, a mesma autenticação e os mesmos seletores estáveis dos screenshots. O resultado é um arquivo leve — formato WebM, da ordem de poucos megabytes para trinta a quarenta segundos —, versionável junto ao conteúdo e incorporável nos artigos.

- Gravar jornadas reais e completas, roteirizadas como o usuário as executaria, e não telas isoladas.
- Fixar viewport, tema, idioma e navegador e desabilitar animações — as mesmas garantias de consistência dos screenshots.
- Obter o ritmo assistível com pausas explícitas entre passos e efeito de digitação, e não com “câmera lenta” global, que multiplica cada ação e faz um vídeo de segundos virar minutos.
- Aguardar elementos concretos na tela, nunca a rede ficar ociosa: aplicações com atualização em tempo real mantêm conexões abertas e a rede nunca “assenta”.
- Em listas paginadas, filtrar pelo nome do item antes de agir; um registro recém-criado pode cair em outra página e não estar visível.
- Usar um nome único por execução para o registro criado, evitando colidir com execuções anteriores e acumular dados de teste.
- Salvar o vídeo depois que o teste conclui, copiando-o do diretório de resultados; salvá-lo de dentro do teste trava a execução.

Overlays de vídeo: cursor e legenda

O Playwright não desenha o cursor do mouse nem legendas no vídeo — ele grava apenas o que está na página. A solução é injetar esses elementos como HTML na própria página: por estarem na tela, entram gravados naturalmente. Um único arquivo reutilizável concentra os dois overlays e serve a qualquer jornada de qualquer produto (compartilhado/playwright/video-overlay.ts).

A legenda é uma faixa fixa no rodapé cujo texto o roteiro troca a cada fase (“Criando um novo projeto”, “Adicionando pessoas ao time”, e assim por diante); uma string vazia a esconde. O cursor é um círculo que acompanha o movimento do mouse e cresce a cada clique, tornando visível onde a ação acontece.

```
// injeta a faixa uma vez; addInitScript reroda a cada navegação
await page.addInitScript(() => {
  const faixa = document.createElement('div');
  faixa.id = '__pw_legenda';
  faixa.style.cssText = 'position:fixed;bottom:28px;left:50%;transform:translateX(-50%);'
    + 'padding:10px 20px;border-radius:10px;background:rgba(17,21,24,.92);color:#fff;'
    + 'z-index:2147483647;opacity:0;transition:opacity .25s:pointer-events:none';
  document.body.appendChild(faixa);
  // ponte entre o teste (Node) e o DOM (navegador)
  window.__setLegenda = (t) => { faixa.textContent = t; faixa.style.opacity = t ? 1 : 0; };
});
// no roteiro, a cada fase:
await legenda(page, 'Adicionando pessoas ao time');
```

Três detalhes que fazem funcionar

- Criar a faixa com addInitScript, e não com uma avaliação pontual: o script reroda a cada nova página, então a legenda não desaparece quando a aplicação navega.
- Combinar pointer-events:none com um z-index altíssimo: a faixa fica acima de tudo, mas não intercepta cliques — se interceptasse, atrapalharia a automação.
- Uma função global na página (window.__setLegenda) é a ponte entre o teste, que roda em Node, e o DOM, que vive no navegador: o teste a chama para trocar o texto.

Execução: um comando dedicado percorre os roteiros de jornada com uma configuração de vídeo própria (viewport ampla, gravação ligada, apenas os specs de jornada) e salva o resultado em projetos/<produto>/static/videos/. Requer a aplicação no ar e a conta de demonstração — as mesmas premissas dos screenshots.

17. Armadilhas conhecidas e verificação de conteúdo

As falhas relatadas a seguir foram observadas em implantação real e não são evidentes durante o planejamento. Antecipá-las reduz retrabalho significativo.

17.1 Verificação de conteúdo produzido com apoio de IA

Ferramentas assistidas por IA aceleram consideravelmente a produção da documentação, mas introduzem um risco específico: o texto resultante é fluente, bem estruturado e aparenta precisão, mesmo quando descreve comportamentos inexistentes. A revisão não pode se limitar à leitura.

Padrões de erro observados

- Funcionalidades inexistentes descritas como disponíveis — por exemplo, uma lista de integrações que não possuíam qualquer implementação no código.
- Componentes existentes declarados como ausentes, resultando em lacunas de documentação sobre partes relevantes do sistema.
- Permissões generalizadas por semelhança de nome de perfil, sem consultar o controle de acesso real.
- Resumos de execução afirmando conclusão de etapas que não foram efetivamente realizadas.

Controle recomendado: Exigir que toda afirmação técnica relevante indique o arquivo de origem, e auditar por amostragem cruzando a documentação com o código. Marcar explicitamente o que é inferência. A auditoria deve verificar existência, e não apenas coerência.

17.2 Consistência entre as vertentes

Documentação técnica e tutoriais são escritos em momentos distintos e tendem a divergir. O caso mais sensível é o de permissões: tutoriais costumam citar perfis de forma genérica, enquanto o controle de acesso real é mais restritivo. O efeito é orientar o usuário a executar uma ação que ele não consegue realizar.

Recomenda-se produzir uma matriz de permissões derivada do controle de acesso implementado e revisar todos os tutoriais contra essa matriz, tratando-a como fonte única de verdade.

17.3 Armadilhas de publicação e ambiente

Estas falhas têm em comum o fato de não produzirem mensagem de erro: tudo aparenta funcionar, e o defeito só é percebido pelo usuário final.

Armadilha	Sintoma e correção
Status 200 enganoso	Aplicações de página única respondem com sucesso a qualquer endereço. Validar por conteúdo e por arquivo com extensão.
Barra final ausente	A página existe, mas a URL sem barra cai na aplicação. Habilitar a barra final no gerador e construir todos os links com ela.
Servidor de desenvolvimento	A documentação em subcaminho não abre em ambiente de desenvolvimento, embora funcione em produção. Validar com o build.
Publicação não propagada	O commit existe, mas a plataforma ainda não reconstruiu. Conferir o painel de publicação antes de investigar o código.

Armadilha	Sintoma e correção
Links relativos a diretórios	Ao habilitar a barra final, links relativos que apontam para pastas passam a resolver errado. Apontar para o arquivo de destino.
Índice de busca desatualizado	A tela de ajuda mostra conteúdo antigo porque o índice não foi regenerado. Encadear a geração no mesmo comando de publicação.

17.4 Lista de verificação antes da publicação

1. Nenhum arquivo de ambiente, sessão ou dependência está versionado.
2. Nenhum caminho local de máquina de desenvolvimento aparece no conteúdo publicado.
3. Todas as funcionalidades descritas existem e estão ativas no produto.
4. A matriz de permissões corresponde ao controle de acesso real.
5. Links internos, imagens e diagramas foram validados pela build.
6. A publicação foi verificada por conteúdo, e não apenas por código de status.
7. Os temas claro e escuro foram revisados em diferentes larguras de tela.
8. Todos os endereços construídos fora do gerador terminam com barra.
9. O índice consumido pela aplicação foi regenerado junto com a publicação.

18. Governança e manutenção

A qualidade da plataforma depende menos da primeira publicação e mais da rotina de manutenção. A documentação deve ser tratada como parte do produto, com responsáveis e critérios objetivos.

Evento	Ação necessária	Responsável sugerido
Nova funcionalidade	Criar ou atualizar arquitetura, tutorial e matriz de cobertura.	Desenvolvimento e produto
Alteração visual	Revisar etapas, rótulos, screenshots e testes visuais.	Frontend e conteúdo
Mudança de permissão	Atualizar a matriz de acesso e todos os tutoriais afetados.	Backend e segurança
Nova integração	Documentar credenciais, fluxo, falhas, limites e operação.	Responsável técnico
Incidente recorrente	Criar troubleshooting com diagnóstico e recuperação.	Suporte e engenharia
Release relevante	Revisar versão, links, cobertura e notas de mudança.	Responsável pela documentação

19. Versionamento da documentação

O gerador permite congelar versões da documentação, criando cópias associadas a releases específicos e um seletor de versão para o leitor. O recurso é útil, porém frequentemente adotado sem necessidade.

Cenário	Recomendação
Software instalado ou distribuído	Adotar versionamento: clientes permanecem em versões diferentes por longos períodos.
API pública com contrato versionado	Adotar versionamento: consumidores dependem de versões específicas.
Aplicação em nuvem com versão única	Evitar versionamento: duplica todo o conteúdo e multiplica a manutenção sem benefício ao leitor.

Quando o versionamento formal não se justifica, alternativas de custo muito menor atendem à necessidade de rastreabilidade: o histórico do repositório, a data de última revisão registrada em cada artigo e as notas de versão do produto.

20. Critérios de qualidade

- O conteúdo corresponde ao comportamento atual do sistema.
- Cada artigo possui público, objetivo e escopo claros.
- Links internos, imagens e diagramas são válidos.
- Tutoriais podem ser executados do início ao fim.
- Nenhum segredo, cookie, dado pessoal ou caminho privado é publicado.
- Linguagem e nomenclatura são consistentes.
- Conteúdo técnico indica limitações e cenários de falha.
- O portal funciona em desktop, tablet, celular e tema escuro.
- A build e as automações principais passam antes da publicação.
- A publicação foi validada no endereço final, por conteúdo.

21. Indicadores de sucesso

Indicador	Como medir
Cobertura	Percentual de módulos com arquitetura, tutorial e evidência visual.
Atualidade	Percentual de artigos revisados dentro do prazo definido.
Confiabilidade	Taxa de sucesso dos testes que validam fluxos documentados.
Uso	Artigos mais acessados, buscas sem resultado e taxa de saída.
Suporte	Redução de dúvidas repetidas e tempo médio de atendimento.
Onboarding	Tempo para novos membros concluírem tarefas essenciais.

22. Riscos e controles

Risco	Controle recomendado
Conteúdo bem apresentado, porém incorreto	Exigir evidência no código e revisão pelo responsável do módulo.

Risco	Controle recomendado
Screenshots com dados reais	Ambiente demonstrativo, mascaramento e revisão de privacidade.
Automação frágil	Seletores estáveis, estados explícitos e ausência de pausas fixas.
Documentação abandonada	Critério de pronto, responsáveis por módulo e lembrete automatizado.
Excesso de imagens	Capturar apenas decisões e resultados difíceis de explicar em texto.
Portal acoplado ao produto	Dependências, build e deploy isolados.
Publicação não verificada	Validar por conteúdo e por arquivo com extensão, não apenas por status.
Exposição de credenciais	Revisar o que será versionado antes do primeiro commit.
Quebra por atualização	Fixar versões, revisar dependências e validar em integração contínua.

23. Escopo mínimo viável

Uma primeira versão útil não precisa cobrir todo o sistema. O escopo inicial deve priorizar os módulos mais utilizados e os fluxos que geram mais dúvidas ou risco operacional.

- Página inicial e mapa da documentação.
- Visão geral da arquitetura e componentes principais.
- Autenticação, perfis e permissões.
- Três a cinco fluxos críticos de uso.
- Configuração, implantação e troubleshooting básico.
- Glossário e matriz inicial de cobertura.
- Automação de login e screenshots das jornadas prioritárias.
- Estratégia de publicação definida e validada no endereço final.

24. Entregáveis

Entregável	Descrição
Portal	Site de documentação com identidade visual, navegação e pesquisa.
Arquitetura	Conjunto de artigos técnicos, diagramas e referências.
Usabilidade	Tutoriais orientados a tarefas e organizados por módulo.
Automação	Roteiros de login, capturas e validações críticas.
Publicação	Estratégia de hospedagem documentada, com procedimento e reversão.
Ajuda no produto	Índice consumido pela tela de ajuda, com busca e filtragem por perfil.
Governança	Critérios de contribuição, revisão e publicação.
Cobertura	Matriz que relaciona módulos, artigos, screenshots e status.

25. Recomendação final

Recomenda-se implantar a plataforma em etapas: fundação, inventário, arquitetura, usabilidade, automação, auditoria e publicação. A ordem é importante porque reduz o risco de produzir uma documentação visualmente atraente, mas desconectada do sistema real.

O gerador de sites deve ser adotado como base do portal e a automação de navegador como mecanismo de validação e produção de evidências. O conteúdo deve viver na instalação compartilhada, versionado à parte e com execução independente do produto. A responsabilidade pela atualização deve fazer parte do processo de entrega de funcionalidades.

Quanto à publicação, recomenda-se avaliar a estratégia logo no início, e não ao final: ela determina parâmetros de configuração e afeta a estrutura do repositório. Quando a hospedagem do produto permitir, o deploy separado oferece o melhor equilíbrio entre isolamento e simplicidade. Quando não permitir, a publicação embutida é uma alternativa legítima, desde que apenas o artefato de build seja incorporado e a verificação seja feita por conteúdo.

Decisão recomendada: Aprovar um piloto com os fluxos mais críticos, estabelecer padrões editoriais, definir a estratégia de publicação e medir cobertura, uso e redução de dúvidas antes de ampliar para todo o produto.

26. Fontes oficiais consultadas

- Docusaurus — Styling and Layout: <https://docusaurus.io/docs/styling-layout>
- Docusaurus — Deployment: <https://docusaurus.io/docs/deployment>
- Docusaurus — Configuration (url e baseUrl): <https://docusaurus.io/docs/configuration>
- Docusaurus — Docs plugin (routeBasePath):
<https://docusaurus.io/docs/api/plugins/@docusaurus/plugin-content-docs>
- Docusaurus — Versioning: <https://docusaurus.io/docs/versioning>
- Docusaurus — Diagrams: <https://docusaurus.io/docs/markdown-features/diagrams>
- Docusaurus — Theme configuration: <https://docusaurus.io/docs/api/themes/configuration>
- Docusaurus — Static assets: <https://docusaurus.io/docs/static-assets>
- Playwright — Library: <https://playwright.dev/docs/library>
- Playwright — Authentication: <https://playwright.dev/docs/auth>
- Playwright — Visual comparisons: <https://playwright.dev/docs/test-snapshots>
- Playwright — Emulation (locale e viewport): <https://playwright.dev/docs/emulation>
- Playwright — Configuration and recording: <https://playwright.dev/docs/test-use-options>
- Docusaurus — trailingSlash: <https://docusaurus.io/docs/api/docusaurus-config#trailingSlash>
- Docusaurus — Sitemap plugin: <https://docusaurus.io/docs/api/plugins/@docusaurus/plugin-sitemap>
- Vite — Public directory e build: <https://vite.dev/guide/assets#the-public-directory>